

Azraq Monte Carlo — complete guide and API reference (single file)

This is the one Markdown document for the product: **HTTP vs engine v0/v1/v2, integration, information flow and validation, all 21 routes** (auth, models, errors, curl examples), **real data / calibration**, and **client FAQ**.

Live: [Swagger UI](#) · `https://monte-carlo.octaloop.dev/openapi.json` · local `http://127.0.0.1:8000`

PDF / document revision: 1.1.0 (2026-05-05). This tracks **changes to this reference export** — it is **not** the `/v1/` REST prefix, and **not** the optional `semver` query default (`1.0.0`) on `POST /v1/shockpack/catalog/register`.

Print PDF: from repo root run `python scripts/build_api_reference_pdf.py` (writes `docs/Azraq_Monte_Carlo_API_Reference.pdf` and a copy `API_Reference.pdf` at repo root). For extra schema narrative and examples beyond OpenAPI, see [API.md](#). A route-only catalogue lives in [ENDPOINTS.md](#) if you prefer a shorter list.

Table of contents

1. [Part I — Product overview: v0 / v1 / v2, real data, FAQ](#)
 2. [Part II — Integration quick guide](#)
 3. [Part III — Information flow, shocks, and validation](#)
 4. [Part IV — API endpoints \(full reference\)](#)
-

Part I — Product overview: v0 / v1 / v2, real data, FAQ

One document for **sales, integration, and validation**. It ties together **what “v0 / v1 / v2” mean in this product, which HTTP routes you call, how live or file-based data feeds the model**, and **typical client questions**.

I.1 The two “versions” (do not confuse them)

Meaning	What it is	Examples
HTTP API prefix	Every business route today lives under <code>/v1/...</code> (plus <code>/ws/v1/...</code> for one WebSocket).	<code>POST /v1/simulate/asset</code>
Engine / model label	Strings in response metadata such as <code>model_version</code> and <code>execution_mode</code> . These describe <i>which code path ran</i> , not the URL prefix.	<code>model_version: "azraq-mc-v1"</code> , <code>execution_mode: "v0_base"</code>

Important: The deterministic “v0” baseline is **not** mounted at `/v0/...`. It is exposed as `POST /v1/simulate/v0/base-case` — the path is still `/v1`, and the `v0` in the path marks the *baseline* operation.

There is **no** separate public `/v2/...` REST prefix. Portfolio “v2” is the `model_version` string for the **joint multi-asset** simulator, still called via `/v1/simulate/portfolio` (or the WebSocket below).

I.2 Engine v0 — deterministic baseline (spreadsheet parity)

Purpose: One scenario, **no Monte Carlo randomness**. Shock vector is zero; you check that the server’s **DSCR / IRR / cashflow-style outputs** match your Excel or internal model for the same `asset` JSON.

Field	Typical value
<code>model_version</code>	<code>azraq-mc-v0</code>
<code>execution_mode</code>	<code>v0_base</code>
HTTP operation	<code>POST /v1/simulate/v0/base-case</code>
Request body	<code>asset</code> only (<code>AssetAssumptions</code>) — same shape you use for MC
Response	<code>BaseCaseResult</code> : <code>metadata</code> + <code>base</code> (point estimates: <code>dscr</code> , <code>irr_annual</code> , <code>annual_revenue</code> , <code>ebitda</code> , etc.)

Supporting APIs (optional around v0): Any `GET /v1/market/...` OR `POST /v1/calibration/preview` are *not required* for v0; they matter when you later build `shockpack` with `dynamic_margins` .

I.3 Engine v1 — single-asset Monte Carlo (main product path)

Purpose: Many correlated random draws → **distributions** of DSCR, covenant breach probability, percentiles, optional attribution, etc.

Field	Typical value
<code>model_version</code>	<code>azraq-mc-v1</code>
<code>execution_mode</code>	<code>adhoc_asset</code> (interactive POST) or <code>scheduled_monitoring</code> (scheduled POST)
Primary HTTP operations	<code>POST /v1/simulate/asset</code> (main), <code>POST /v1/simulate/scheduled/asset</code> (same maths + optional on-disk JSON snapshot)

What you send: `asset` + `shockpack` (`ShockPackSpec`), OR `shockpack_catalog_entry_id` (with optional inline patch), plus optional flags (`performance_profile` , `include_attribution` , full-stack options on `asset` , etc.). Full schema: OpenAPI `/openapi.json` or [API.md](#).

I.4 Engine v2 — portfolio (joint shocks across ≥2 assets)

Purpose: One shared random world: scenario *k* uses the **same** shock draw for every asset in the list, so correlations across projects are meaningful.

Field	Typical value
<code>model_version</code>	<code>azraq-mc-v2-portfolio</code>
<code>execution_mode</code>	<code>portfolio_joint</code>
HTTP operation	<code>POST /v1/simulate/portfolio</code>
Real-time operation	WebSocket <code>WS /ws/v1/simulate/portfolio</code> (same joint simulation; can stream progress)

Requirement: At least two assets with **distinct** `asset_id` values.

I.5 Which API for which job (quick map)

Client need	Call
“Does your math match our spreadsheet?”	<code>POST /v1/simulate/v0/base-case</code>
“What’s our tail risk / breach % for one project?”	<code>POST /v1/simulate/asset</code>
“Run again every week and store JSON on disk.”	<code>POST /v1/simulate/scheduled/asset</code>
“Two or more projects, same shock world.”	<code>POST /v1/simulate/portfolio</code> or <code>WS /ws/v1/simulate/portfolio</code>
“Prove Yahoo / our file filled volatilities.”	<code>POST /v1/calibration/preview</code> , then check <code>calibration_trace / run metadata.margin_calibration_trace</code>
“Inspect copper / VIX history before we trust a symbol.”	<code>GET /v1/market/yfinance/{symbol}/history</code>
“Reuse a signed-off shockpack.”	<code>POST /v1/shockpack/catalog/register</code> , then simulate with <code>shockpack_catalog_entry_id</code>
“Compare two saved runs.”	<code>POST /v1/snapshots/diff/asset-metrics</code>

The **numbered route inventory** and **per-endpoint reference** are in [Part IV](#) below.

I.6 Real data vs the Monte Carlo core (plain language)

The Monte Carlo core does not need a live market feed to run. It needs:

1. `asset` — *your* deal assumptions (no randomness).
2. `shockpack` — *how* randomness enters: scenario count, seed, `margins` (volatilities / dispersion), **correlation**, copula, optional time structure.

Where “real data” enters: It **calibrates** those margins (and can be audited), via `dynamic_margins` on the shockpack:

Source	Mechanism	Typical use
Yahoo Finance	<code>dynamic_margins.yahoo_finance</code> → annualised log vol from closes; <code>GET /v1/market/yfinance/...</code> for manual inspection	Commodities, equity indices, some proxies
ECB	<code>GET /v1/market/ecb/eur-usd</code> reads public ECB daily XML	FX spot sanity / storytelling (MC still uses your shockpack params)
Your HTTP API	<code>dynamic_margins.http</code> — trusted URL returns JSON margins	Internal ETL, SOFR/SONIA proxies you host
File	<code>dynamic_margins.file</code> — JSON or Excel with margin columns	Signed-off vol sheets from risk team

Order of application (product rule): `file` → `http` → each `yahoo_finance` binding (later steps can override the same margin keys). After materialisation, resolved numbers sit in `margins` and `dynamic_margins` is cleared on the resolved spec.

Proof for auditors / clients: Use `POST /v1/calibration/preview` to see `resolved_shockpack` without paying for MC; on a full run, inspect `metadata.margin_calibration_trace`. The machine-readable list of stress-source ideas is `GET /v1/calibration/stress-data-catalog`.

Security note: File paths and HTTP URLs run **inside the API process**. In production, restrict to **trusted** paths and URLs (SSRF / file-read risk).

I.7 Recommended validation order (before trusting tails)

1. Build `asset` from your systems.
2. Build `shockpack` (start with manual `margins` if needed).
3. Optional: `POST /v1/calibration/preview` if using `dynamic_margins`.
4. `POST /v1/simulate/v0/base-case` — must align with your spreadsheet.
5. `POST /v1/simulate/asset` — interpret `SimulationResult.metrics` (and optional attribution as **indicative** narrative, not a regulatory sign-off by itself).

More detail: [Part III](#).

I.8 Client questions (FAQ)

Q: Why is everything /v1 if you talk about v0 and v2?

A: `/v1` is the **REST version of the HTTP contract**. `v0` / `v1` / `v2` in conversation usually mean `model_version` / `execution_mode` (deterministic baseline vs single-asset MC vs joint portfolio).

Q: Is v0 deprecated?

A: No. It is the **recommended sanity check** before Monte Carlo.

Q: Do you stream live prices into the simulation?

A: Not as a continuous feed. **Live (or file) data can refresh volatility parameters** via `dynamic_margins`; the simulation itself draws random scenarios from the **parameterised** shock model you define.

Q: Will Yahoo Finance always work?

A: It depends on `yfinance` and Yahoo's availability. For production calibration, many clients prefer `http` or `file` from their own curated data.

Q: What does reproducibility mean?

A: Fix `asset`, `shockpack.seed`, `shockpack.n_scenarios`, and the resolved `margins`. The same inputs should yield the same `SimulationResult` (subject to documented floating-point / dependency behaviour).

Q: Is the service stateless?

A: Each `POST /v1/simulate/asset` is independent unless **you** choose catalogue ids or snapshot endpoints to reuse stored artefacts.

Q: How do we explain “shocks” to a non-quant?

A: Shocks are **random draws** that scale risk factors (revenue, capex, opex, rate, etc.) according to **volatility and correlation** you set — not a list of hand-picked “–20% events.” See [Part III §1](#).

Q: What is full-stack mode?

A: Optional **12-factor** shock design aligned with `GET /v1/catalog/full-stack-factors` and `asset.full_stack.enabled`. Use only when your `shockpack` matches that dimensionality.

Q: What headers might we need?

A: If `AZRAQ_API_KEY` is set: `X-API-Key` on all routes except `GET /health`. Optional: `X-Azraq-User-Id` (audit

label), `X-Azraq-Tenant-Id` (catalogue), `X-Azraq-Catalog-Role` (promote). Details: [Part IV §1](#).

Q: Where do “layer versions” appear?

A: Audit metadata can bundle semantic versions for shock/transform/metrics/impact layers (`layer_versions` in code). These support **incremental recomputation and traceability**, not separate HTTP paths.

I.9 Hosted and local URLs

Environment	Base	Docs
Hosted (example)	<code>https://monte-carlo.octaloop.dev</code>	<code>/docs</code> , <code>/openapi.json</code>
Local dev	<code>http://127.0.0.1:8000</code>	same

Part II — Integration quick guide

Short reference for plugging **your data** into the Monte Carlo API. **Every route** is also documented in [Part IV](#). Narrative + extra examples: [API.md](#).

II.1 Base URL

Use your deployed host, e.g. `https://api.example.com` or local `http://127.0.0.1:8000`.

II.2 Primary call (single project)

Method / path	<code>POST /v1/simulate/asset</code>
Body	JSON <code>AdhocSimulationRequest</code> : at minimum <code>asset</code> + <code>shockpack</code>
Response	<code>SimulationResult</code> : <code>metadata</code> , <code>metrics</code> , optional <code>attribution</code> , etc.

Health check (no API key on typical setups): `GET /health`

II.3 Authentication & headers

Env / config	Client header
If <code>AZRAQ_API_KEY</code> is set on the server	Send <code>X-API-Key: <same value></code> on requests that require it (<code>GET /health</code> is usually public).

Optional audit labels (not login):

- `X-Azraq-User-Id`
- `X-Azraq-Tenant-Id` (used for some catalogue routes)

II.4 “Reset” and state

The service is **stateless**: each `POST /v1/simulate/asset` is independent. There is **no server-side session** to clear. To “reset,” **discard the previous response** and **POST new JSON**.

To **persist** full JSON snapshots on the server for later diffing, see `POST /v1/simulate/scheduled/asset` in [Part IV](#).

II.5 Minimal request shape

```
{
  "shockpack": {
    "shockpack_id": "your-pack-id",
    "seed": 42,
    "n_scenarios": 2500,
    "sampling_method": "monte_carlo",
    "margins": {
      "revenue_log_mean": 0,
      "revenue_log_sigma": 0.08,
      "capex_log_mean": 0,
      "capex_log_sigma": 0.06,
      "opex_log_mean": 0,
      "opex_log_sigma": 0.05,
      "rate_shock_sigma": 0.005
    }
  },
  "asset": {
    "asset_id": "your-asset-id",
    "assumption_set_id": "your-label",
    "horizon_years": 15,
    "base_revenue_annual": 42000000,
    "base_opex_annual": 18000000,
    "utility_opex_annual": 0,
    "initial_capex": 280000000,
    "equity_fraction": 0.35,
    "tax_rate": 0,
    "financing": {
      "debt_principal": 182000000,
      "interest_rate_annual": 0.065,
      "loan_term_years": 18,
      "covenant_dscr": 1.2
    }
  },
  "include_attribution": false,
  "performance_profile": "interactive"
}
```

`performance_profile`: `interactive` caps scenario count for responsiveness; `standard` / `deep` allow larger `n_scenarios` (see OpenAPI / [API.md](#)).

II.6 What to read from the response

Your product need	Typical field under <code>metrics</code>
DSCR distribution	<code>dscr</code> (p05, p50, p95, ...)
IRR distribution	<code>irr_annual</code>
P(covenant breach)	<code>covenant_breach_probability</code>
Tail / VaR-style IRR	<code>var_irr_95</code> , <code>cvar_irr_95</code> (when populated)
Build cost distribution	<code>total_capex</code> (when present)

Exact shapes are in **OpenAPI** (`/openapi.json`).

II.7 Mapping checklist (your data → API)

Your field / concept	API location
Project / case id	<code>asset.asset_id</code> , <code>asset.assumption_set_id</code>
Model horizon	<code>asset.horizon_years</code>
Revenue, opex, capex	<code>asset.base_revenue_annual</code> , <code>base_opex_annual</code> , <code>initial_capex</code>
Equity share	<code>asset.equity_fraction</code> (decimal, e.g. 0.35)
Debt, coupon, term, DSCR floor	<code>asset.financing: debt_principal</code> , <code>interest_rate_annual</code> (decimal), <code>loan_term_years</code> , <code>covenant_dscr</code>
Shock count / reproducibility	<code>shockpack.n_scenarios</code> , <code>shockpack.seed</code>
Volatility / margins	<code>shockpack.margins</code> (and optional <code>dynamic_margins</code> for Yahoo/file/HTTP calibration — API.md)

II.8 Optional next steps

- **Catalogue-managed shock packs:** register a base spec, then call with `shockpack_catalog_entry_id` + optional inline `shockpack patch` — see [API.md](#).
- **Portfolio:** `POST /v1/simulate/portfolio` for multiple correlated assets.
- **Codegen:** import `/openapi.json` into your stack (OpenAPI generators, Postman, etc.).

II.9 Other docs in this repo

Doc	Use
This file (<code>Azraq_Monte_Carlo_API_Reference.md</code>)	Versions, integration, flow, validation, full route reference in one place
ENDPOINTS.md	Same route catalogue as Part IV (kept for PDF script + short links)
API.md	Longer narrative, <code>response_help</code> , security notes, shared types
PROJECT OVERVIEW.md	Product-level engine description
SCENARIO LAB NON TECH GUIDE.md	Scenario Lab UI (<code>/app/</code>)

Part III — Information flow, shocks, and validation

This section is for teams who can call the API but need a **clear picture of what travels where**, what “**shocks**” mean in this product, and **how to prove** that your own numbers actually move the results.

III.1 Two separate layers (this is the mental model)

Layer	JSON object	Role
Project economics	<code>asset</code> (<code>AssetAssumptions</code>)	<i>Your deal:</i> revenue, opex, capex, horizon, financing, covenant floor, optional transforms / full-stack flags. No randomness here —these are the baseline inputs the engine stresses.
Uncertainty model	<code>shockpack</code> (<code>ShockPackSpec</code>)	<i>How the world wobbles:</i> how many scenarios, seed, sampling method, which risk factors (<code>factor_order</code>), how volatile each is (<code>margins</code>), how they move together (<code>correlation</code> , <code>copula</code>), optional paths over time (<code>time_grid</code>), optional calibration from files/HTTP/Yahoo (<code>dynamic_margins</code>).

Monte Carlo = draw many correlated random **shock vectors** → for each draw, recompute coverage / cashflow-style metrics → summarise as distributions (percentiles, breach probability, etc.).

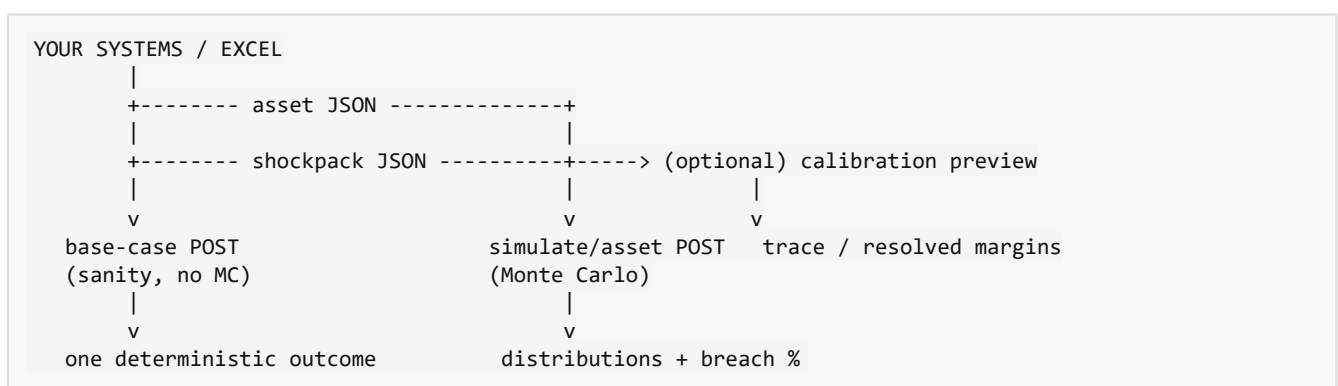
Shocks are not “events” you type in one-by-one (like “–20% revenue”). They are **parameters of random draws**: e.g. lognormal volatility on revenue (`revenue_log_sigma`) plus correlation with capex, opex, and rate. A **stress** in the colloquial sense is achieved by **changing those parameters** (wider sigmas, different correlation, or deterministic bumps via `factor_transforms` on the asset—see §III.4).

III.2 End-to-end flow (what to call, in order)

Start here — five steps (no diagram required)

1. Build `asset` (`AssetAssumptions`) from your spreadsheet or systems — the project as numbers you believe today.
2. Build `shockpack` (`ShockPackSpec`) — how many scenarios, seed, volatilities, correlation, optional `dynamic_margins` (Yahoo / file / HTTP).
3. **Optional:** `POST /v1/calibration/preview` with that shockpack — confirms external data filled `margins` without running MC (see `calibration_trace`).
4. `POST /v1/simulate/v0/base-case` with `asset` **only** — one deterministic row; must match your spreadsheet logic before you trust random runs.
5. `POST /v1/simulate/asset` with `asset` + `shockpack` — full Monte Carlo; read `SimulationResult.metrics` .

Same idea as a simple picture (read top → bottom)



Optional later (only if you need them)

- **Several projects together:** `POST /v1/simulate/portfolio` (2+ assets, one shared shock world).
- **Save and compare runs:** `POST /v1/snapshots/save` and `GET /v1/snapshots/diff/asset-metrics` .
- **Reuse a stored shockpack:** `POST /v1/shockpack/catalog/register` then call simulate with `shockpack_catalog_entry_id` .

- **Browse stress-source ideas:** `GET /v1/calibration/stress-data-catalog`; **inspect** a series: `GET /v1/market/yfinance/{symbol}/history` (does not run MC by itself).

Endpoint one-liners

- `POST /v1/simulate/v0/base-case`: same `asset` shape as MC, **no** random draws — align with a spreadsheet first.
- `POST /v1/calibration/preview`: turns `dynamic_margins` into concrete `margins`; no simulation. Use `calibration_trace` / run `metadata.margin_calibration_trace` to prove real data landed.
- `POST /v1/simulate/asset`: main MC — `asset` + `shockpack` → `SimulationResult`.

III.2a `POST /v1/simulate/asset` — full server flow (flowchart)

Who this is for: anyone who wants to see what happens **after** the HTTP POST, in one picture.

Flowchart (read top → bottom)

```

CLIENT
|
| HTTP POST /v1/simulate/asset
| JSON body: AdhocSimulationRequest
|   - asset (required)
|   - shockpack OR shockpack_catalog_entry_id (or both: catalogue base + patch)
|   - optional: performance_profile, include_attribution, ...
v
+-----+
| API key check (if | Header X-API-Key when server requires it
| AZRAQ_API_KEY set) |
+-----+
v
+-----+
| Resolve final Shock pack |
+-----+
|
| If catalogue id in body:
|   -> load stored ShockPackSpec
|   -> if inline shockpack too: patch only fields you sent
| Else:
|   -> inline shockpack required (full spec in request)
|
v
+-----+
| run_adhoc_asset_simulation |
+-----+
v
+-----+
| Apply performance_profile | May cap n_scenarios (e.g. interactive UI)
+-----+
v
+-----+
| materialize_shockpack_ | file / HTTP / Yahoo -> margins;
| margins | trace -> metadata.margin_calibration_trace
+-----+
v
+-----+
| get_or_build_shock_array | Build random draws Z (or reuse cache)
+-----+
v
+-----+
| financial_impact | Each scenario: shocks + asset -> outcomes
| (optional pipeline cache | Skip recompute if same fingerprint hit
| may return cached paths) |
+-----+
v
+-----+
| build_financial_metrics | Distributions, covenant_breach_probability, ...
| (+ full_stack metrics | If asset.full_stack enabled and layer present
| when applicable) |
+-----+
v
+-----+
| Optional attribution | If include_attribution (and advanced flags)
+-----+
v
+-----+
| Assemble SimulationResult | metadata, metrics, attribution?, extensions?
| audit_simulation(...) |
+-----+
v
HTTP 200 + JSON SimulationResult

```

Explain the flowchart (plain language)

1. **You send one request** with the **project** (`asset`) and how to draw risk (`shockpack` and/or a **catalogue id** that points at a saved shock pack).
2. **Resolve** picks the **final shock recipe**: catalogue row first, then any fields you put inline **fill in or override** missing pieces. You cannot omit both catalogue and inline pack.
3. **Performance profile** can **reduce** scenario count for speed (`interactive` / `standard`); `deep` does not cap.
4. **Materialize margins** runs **before** random draws: if the shock pack asked for Yahoo/file/HTTP calibration, those numbers are merged into `margins` now. Check `metadata.margin_calibration_trace` to prove what happened.
5. **Shock array** builds (or loads from cache) the correlated random `z` for all scenarios.
6. **Financial impact** walks every scenario through the model (or reuses cached outcomes when the pipeline cache hits).
7. **Metrics** turns raw paths into **percentiles**, **breach probability**, optional tail stats, optional **full_stack** block.
8. **Attribution** (only if requested) adds “**what drove the bad tail**” style diagnostics — indicative, not a bank sign-off.
9. **Response** is `SimulationResult` JSON; the server also **writes an audit row** for the run.

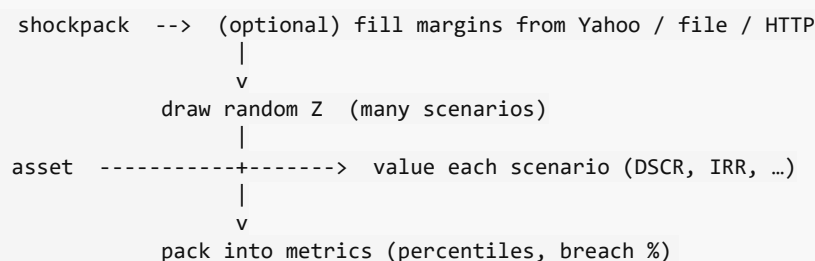
One line: `POST /v1/simulate/asset` = **merge shock recipe** → **calibrate margins** → **draw shocks** → **value the asset many times** → **summarise** → **return JSON**.

III.3 What happens inside one Monte Carlo run (simplified)

Relation to §III.2a: The numbered steps below are the **inside** of the `run_adhoc_asset_simulation` box in §III.2a (from “materialize margins” through “metrics”). §III.2a adds the **HTTP, resolve shock pack**, and **response/audit** wrapper.

This section is **one request on the server** — not the order you build JSON in your team.

One-line picture



Steps (read 1 → 4)

1. **Lock in margins (optional calibration):** If `dynamic_margins` is set, the service **merges** file / HTTP / Yahoo into `margins`, then drops `dynamic_margins` from the resolved spec. Proof: `margin_calibration_trace` on the run when used.
2. **Draw random shocks:** Build `Z` (correlated normals / Student-t scaling), size from `n_scenarios`, `factor_order`, `correlation`, `copula`, optional `time_grid`.

3. **Map shocks to cash drivers:** Apply optional `asset.factor_transforms`, then lognormal / rate rules from `margins` so each scenario has stressed revenue, capex, opex, rate paths.
4. **Summarise:** Each scenario yields DSCR, IRR, etc.; the API returns percentiles and `covenant_breach_probability` in `metrics`.

Code names (only if you grep the repo): `materialize_shockpack_margins` → `get_or_build_shock_array` → `financial_impact`.

So: `margins` **control how wide** the distributions are; `correlation` controls **joint bad years**; `asset.*` controls **the size of the machine** being shocked.

III.4 Shock-related fields and what they do to outputs

Input (typical)	Effect on the run
<code>n_scenarios</code>	More scenarios → smoother percentiles; law of large numbers (diminishing noise).
<code>seed</code>	Reproducibility: same seed + same spec → same Z (subject to performance/cache notes in API docs).
<code>sampling_method</code>	<code>monte_carlo</code> vs <code>latin_hypercube</code> / <code>sobol</code> — changes how space is filled, not the marginal distributions.
<code>factor_order</code> + <code>correlation</code>	Must be square and same length. Higher positive correlation between revenue and opex (for example) moves joint tail risk.
<code>margins.revenue_log_sigma</code> (etc.)	Wider log σ → wider revenue (and downstream DSCR / IRR) distribution; breach probability often rises if downside matters for covenant.
<code>margins.rate_shock_sigma</code>	Volatility of additive shocks to the annual interest rate (decimal σ); increases rate path dispersion.
<code>copula</code> / <code>t_degrees_freedom</code>	Gaussian vs heavier tails on the dependence structure.
<code>time_grid</code>	Multi-period Z: e.g. min DSCR over life vs single-period snapshot (see API narrative).
<code>asset.factor_transforms</code>	Deterministic scaling of shock indices or level multipliers (e.g. stress capex level) — useful when you want a known wedge on top of stochastic draws.
<code>macro_regime</code>	Label for audit / catalogue; does not by itself change maths unless you pair it with different specs per regime.

External “real data” does **not** stream into every scenario. It is used to **set or update** the `margins` (and you verify that via **preview** or `margin_calibration_trace`).

III.5 Step-by-step: prove your data affects the outcome

Do these in order the first time you integrate.

Step A — Health and contract

1. `GET /health` — environment up.
2. Open `/openapi.json` or [Part IV](#) — authoritative list of fields and types.

Step B — Deterministic baseline (asset only)

1. `POST /v1/simulate/v0/base-case` with **only** your `asset` (same shape as in MC).
2. Check `base` metrics against your internal spreadsheet for the same inputs. If this fails, fix `asset` mapping before touching shocks.

Step C — One-dimensional sensitivity (asset)

For each critical field (`base_revenue_annual` , `initial_capex` , `financing.interest_rate_annual` , `financing.covenant_dscr` , ...):

1. Change **one** field by a **small, known** amount (e.g. +5% revenue).
2. Re-run base-case; confirm the output moves in the **direction you expect** (e.g. higher revenue → better coverage if nothing else breaks validation).

This proves **your economic payload** is wired.

Step D — Shock width sensitivity (shockpack)

Fix `asset` and `seed`. Run `POST /v1/simulate/asset` twice:

1. **Low vol:** small `revenue_log_sigma` , `capex_log_sigma` , `opex_log_sigma` , `rate_shock_sigma` .
2. **High vol:** multiply those sigmas by e.g. **2×** (keep correlation identical).

Expect: breach probability and dispersion of DSCR / IRR **increase** with higher σ . If nothing moves, you are likely not hitting the simulate route you think you are, or metrics are cached from an identical fingerprint—compare `metadata.run_id` and request bodies.

Step E — Calibration path (optional)

1. Put a minimal margin patch in JSON or Excel; reference it in `dynamic_margins.file` , or configure `dynamic_margins.yahoo_finance` with a liquid symbol.
2. `POST /v1/calibration/preview` with the same `shockpack` you intend to use.
3. Read `resolved_shockpack.margins` and `calibration_trace` . Confirm numbers match your file / Yahoo-derived expectation.
4. Run `POST /v1/simulate/asset` and confirm `metadata.margin_calibration_trace` is populated and consistent with preview.

Step F — Portfolio (if applicable)

1. Two assets, shared `shockpack` , `POST /v1/simulate/portfolio` .
2. Toggle correlation between a macro factor shared across assets (via `correlation`) and confirm **portfolio** breach / joint tail metrics respond.

III.6 Short “video” outline (e.g. 8-minute screen recording)

If you record a walkthrough (Loom, Teams, etc.), this sequence matches **§III.2** above and builds confidence:

1. **0:00–0:45** — Show `/docs + openapi.json` ; say “two blocks: `asset` + `shockpack` ”.
2. **0:45–2:30** — Base-case POST: paste minimal `asset` , show response vs a spreadsheet row.
3. **2:30–4:30** — Same `asset` , two MC runs: low vs high `*_log_sigma` ; show `covenant_breach_probability` and DSCR percentiles.
4. **4:30–6:30** — Calibration preview: `dynamic_margins` with a tiny JSON file or one Yahoo binding; show resolved margins + trace.
5. **6:30–8:00** — Full simulate with trace; point to `response_help` in JSON for non-technical readers.

III.7 One-line summary for stakeholders

You supply a deterministic project model (`asset`) and a stochastic specification (`shockpack`); the API returns distributions of financial outcomes. External series are optional helpers to set volatility parameters—verify them with calibration preview and `margin_calibration_trace` , then validate behaviour with base-case checks and controlled σ sensitivity.

Part IV — API endpoints (full reference)

Hosted (Swagger UI — try requests in the browser): <https://monte-carlo.octaloop.dev/docs#/>

Same deployment: OpenAPI JSON — <https://monte-carlo.octaloop.dev/openapi.json> · optional investor dashboard — <https://monte-carlo.octaloop.dev/app/> (if enabled).

Local dev: <http://127.0.0.1:8000> (same paths: `/docs` , `/openapi.json` , `/app/`).

Machine-readable contract: `GET /openapi.json` . **Interactive UI:** `GET /docs` (Swagger).

Shorter route-only catalogue: [ENDPOINTS.md](#) (kept for diff-friendly edits; this file is the full narrative + reference).

IV.o Contents (Part IV)

1. [Complete route inventory](#)
2. [Authentication & headers](#)
3. [Shared JSON models](#)
4. [How to send data and read results](#)
5. [Endpoint reference](#)
6. [Errors](#)
7. [Live deployment, examples, and response detail](#)

IV.o Complete route inventory

Application API (implemented in `azraq_mc/api.py`)

There are **20 HTTP operations** and **1 WebSocket** — **21** interfaces total.

#	Method	Path	Summary
1	GET	<code>/health</code>	Liveness (no API key).
2	GET	<code>/v1/market/yfinance/{symbol}/history</code>	Yahoo OHLCV (query <code>period</code>).
3	GET	<code>/v1/market/yfinance/{symbol}/returns</code>	Yahoo simple returns.
4	GET	<code>/v1/market/ecb/eur-usd</code>	ECB EUR/USD spot.
5	GET	<code>/v1/calibration/stress-data-catalog</code>	Stress-source catalogue.
6	GET	<code>/v1/catalog/full-stack-factors</code>	12-factor full-stack list.
7	POST	<code>/v1/calibration/preview</code>	Resolve <code>dynamic_margins</code> → <code>margins</code> (no MC).
8	POST	<code>/v1/simulate/scheduled/asset</code>	MC + optional snapshot file.

#	Method	Path	Summary
9	POST	/v1/simulate/asset	Main single-asset Monte Carlo.
10	POST	/v1/simulate/v0/base-case	Deterministic baseline (body = <code>AssetAssumptions</code>).
11	POST	/v1/simulate/portfolio	Joint MC, ≥ 2 assets.
12	POST	/v1/shockpack/catalog/register	Save shock pack $\rightarrow$ <code>entry_id</code> .
13	POST	/v1/shockpack/catalog/{entry_id}/promote	Change <code>promotion_tier</code> .
14	GET	/v1/shockpack/catalog/{entry_id}	Fetch catalogue row + <code>spec</code> .
15	GET	/v1/shockpack/catalog	List catalogue entries.
16	POST	/v1/shockpack/export/npz	Write shock <code>Z</code> to <code>.npz</code> on disk.
17	POST	/v1/snapshots/save	Save prior result JSON.
18	GET	/v1/snapshots/list	List snapshot paths.
19	GET	/v1/audit/runs	Recent audit rows (<code>limit</code>).
20	POST	/v1/snapshots/diff/asset-metrics	Diff two asset <code>SimulationResult</code> files.
21	WebSocket	/ws/v1/simulate/portfolio	Portfolio MC + optional progress.

Anything not in this table is **not** a custom business route (see below).

Also served (FastAPI + static UI)

These are automatic or mounted by the app, not separate “risk” APIs:

Kind	Path	Purpose
OpenAPI schema	GET /openapi.json	Machine-readable spec (codegen, Postman).
Swagger UI	GET /docs	Try-it-out browser UI.
ReDoc	GET /redoc	Alternate docs UI (if enabled by FastAPI defaults).
Static dashboard	GET /app/...	Investor UI (<code>frontend/</code>), only if that folder exists.

IV.1 Authentication & headers

Condition	Client behaviour
Server has <code>AZRAQ_API_KEY</code> set	Send header <code>X-API-Key: <same value></code> on all routes except <code>GET /health</code> (health is public).
Optional audit	<code>X-Azraq-User-Id</code> — label stored with runs (not authentication).
Catalogue / tenant	<code>X-Azraq-Tenant-Id</code> — filters or enforces tenant on some catalog <code>GET</code> routes.
Promoting shock packs	<code>POST /v1/shockpack/catalog/{entry_id}/promote</code> needs <code>X-Azraq-Catalog-Role</code> in the allow-list (<code>AZRAQ_CATALOG_PROMOTER_ROLES</code>), unless that env is empty (local dev).

WebSocket: if an API key is required, pass header `x-api-key` on the handshake (see portfolio WS below).

IV.2 Shared JSON models

These names match **Pydantic** / **OpenAPI** schemas in the repo. Field-level detail: [/docs](#) or [API.md](#) § “Shared types”.

Model	Used for
<code>FinancingAssumptions</code>	<code>debt_principal</code> , <code>interest_rate_annual</code> (decimal), <code>loan_term_years</code> , <code>covenant_dscr</code>
<code>AssetAssumptions</code>	Project economics + <code>financing</code> + optional <code>utility_opex_annual</code> , <code>factor_transforms</code> , <code>full_stack</code> , ...
<code>ShockPackSpec</code>	<code>shockpack_id</code> , <code>seed</code> , <code>n_scenarios</code> , <code>sampling_method</code> , <code>margins</code> , optional <code>dynamic_margins</code> , <code>correlation</code> , <code>factor_order</code> , ...
<code>AdhocSimulationRequest</code>	<code>asset</code> + <code>shockpack</code> and/or <code>shockpack_catalog_entry_id</code> + optional attribution / <code>performance_profile</code>
<code>ScheduledAssetRequest</code>	Same as <code>adhoc</code> + <code>label</code> , <code>persist</code> , <code>model_version</code>
<code>PortfolioSimulationRequest</code>	<code>portfolio_id</code> , <code>portfolio_assumption_set_id</code> , <code>assets</code> (array, ≥ 2), <code>shockpack</code> or catalogue id
<code>SimulationResult</code>	<code>metadata</code> , <code>metrics</code> , optional <code>attribution</code> , <code>full_stack</code> , <code>extensions</code>
<code>BaseCaseResult</code>	Deterministic run: <code>metadata</code> , <code>base</code> (point estimates)
<code>PortfolioSimulationResult</code>	<code>metadata</code> , <code>per_asset[]</code> , <code>portfolio</code>
<code>CalibrationPreviewResponse</code>	<code>resolved_shockpack</code> , <code>calibration_trace</code>
<code>ScheduledAssetResponse</code>	<code>result</code> (<code>SimulationResult</code>), <code>snapshot_path</code>

Most JSON responses also include `response_help` (plain-language hints).

IV.3 How to send data and read results

Single-project Monte Carlo (typical integration)

1. **Build** an `AssetAssumptions` object from your systems (revenue, opex, capex, debt, rates, covenant, horizon, ids).
2. **Build** a `ShockPackSpec` (at least `shockpack_id`, `seed`, `n_scenarios`, `sampling_method`, and usually `margins` or `dynamic_margins` for calibration).
3. **POST** `/v1/simulate/asset` with body `{ "asset": {...}, "shockpack": {...}, ... }`.
4. **Read** 200 JSON `SimulationResult`: use `metrics.dscr`, `metrics.irr_annual`, `metrics.covenant_breach_probability`, tail fields, etc., and `metadata` for `run_id`, timing, seed.

State: there is **no session**. Each POST is independent. To “reset,” send a new body.

Optional: validate calibration only (no MC)

POST `/v1/calibration/preview` with a full `ShockPackSpec` (e.g. with `dynamic_margins.yahoo_finance`). Response gives `resolved_shockpack` with concrete `margins` — paste that into simulate.

Optional: save results for diffing

- **POST** `/v1/simulate/scheduled/asset` with `persist: true` → get `snapshot_path`.
- Or **POST** `/v1/snapshots/save` with a prior `SimulationResult` in `result`.

Then **POST** `/v1/snapshots/diff/asset-metrics` with `before_path` / `after_path`.

IV.4 Endpoint reference

Legend: **Path** / **Query** / **Body** = where parameters go. **Auth** = needs `X-API-Key` when server key is set (except `/health`).

GET `/health`

Purpose	Liveness: server responds.
Parameters	None.
Response	{ "status": "ok", "response_help": { ... } }
Usage	Monitoring or pre-flight check. No model run.
Auth	None.

GET `/v1/market/yfinance/{symbol}/history`

Purpose	Raw OHLCV history from Yahoo (inspection / reporting).
Path	<code>symbol</code> — ticker, e.g. <code>HG=F</code> .
Query	<code>period</code> — <code>1d</code> , <code>5d</code> , <code>1mo</code> , <code>3mo</code> , <code>6mo</code> , <code>1y</code> , <code>2y</code> , <code>5y</code> , <code>10y</code> , <code>ytd</code> , <code>max</code> (default <code>1y</code>).
Response	{ "symbol", "period", "data": [...] } + <code>response_help</code> . 404 if no data; 501 if <code>yfinance</code> missing.
Usage	Does not run Monte Carlo. For feeding vol into the engine use <code>dynamic_margins.yahoo_finance</code> or <code>/v1/calibration/preview</code> .
Auth	Optional key.

GET `/v1/market/yfinance/{symbol}/returns`

Purpose	Close-to-close simple returns series (human review).
Path	<code>symbol</code> .
Query	<code>period</code> — same set as history.
Response	{ "symbol", "period", "returns": [{"date","return"},...], "n" } + <code>response_help</code> . 404 / 501 as above.
Usage	Exploratory; internal calibration uses log returns + annualisation.
Auth	Optional key.

GET `/v1/market/ecb/eur-usd`

Purpose	ECB daily EUR/USD reference from public XML.
Parameters	None.
Response	Rate fields + <code>response_help</code> . 502 if feed fails.

Usage	FX sanity checks; wire into dynamic_margins via your own JSON/file/http if needed.
Auth	Optional key.

GET /v1/calibration/stress-data-catalog

Purpose	Catalogue of stress / data sources and integration hints (Appendix J–style).
Parameters	None.
Response	{ "sources", "integration_overview", "response_help" }.
Usage	Discover URLs and how they map to dynamic_margins .
Auth	Optional key.

GET /v1/catalog/full-stack-factors

Purpose	Ordered factor_order + metadata for 12-factor full-stack mode.
Parameters	None.
Response	{ "factor_order", "risk_factors", "response_help" }.
Usage	Required before asset.full_stack + matching 12-factor ShockPackSpec .
Auth	Optional key.

POST /v1/calibration/preview

Purpose	Resolve dynamic_margins (file / http / Yahoo) into numeric margins without drawing scenarios.
Body	ShockPackSpec (JSON) — same shape as shockpack on simulate.
Response	CalibrationPreviewResponse : resolved_shockpack , calibration_trace .
Usage	Governance: confirm resolved_shockpack.margins before expensive /v1/simulate/asset .
Auth	Optional key.

POST /v1/simulate/v0/base-case

Purpose	Deterministic single path (no random shocks). Fast tie-out vs spreadsheet.
Body	AssetAssumptions as root JSON (not wrapped in { "asset": ... }).
Response	BaseCaseResult : metadata (execution_mode : "v0_base"), base point estimates (dscr , irr_annual , ...).
Usage	Baseline only — not tail risk.
Auth	Optional key.

POST /v1/simulate/asset

Purpose	Main Monte Carlo for one asset — DSCR/IRR distributions, breach probability, etc.

Body	AdhocSimulationRequest : asset (required); shockpack and/or shockpack_catalog_entry_id ; optional include_attribution , include_advanced_attribution , attribution_tail_fraction , performance_profile .
Response	SimulationResult : metadata , metrics , optional attribution , full_stack , extensions .
Usage	Primary integration: send economics + shock spec, receive metrics for dashboards or storage.
Auth	Optional key.

POST /v1/simulate/scheduled/asset

Purpose	Same maths as /v1/simulate/asset, optional on-disk JSON snapshot for monitoring / diff.
Body	ScheduledAssetRequest : all AdhocSimulationRequest fields + label , persist (default true), model_version .
Response	ScheduledAssetResponse : result (SimulationResult), snapshot_path or null.
Usage	Recurring runs; pair with /v1/snapshots/diff/asset-metrics. Snapshot root: AZRAQ_SNAPSHOT_DIR or default data/snapshots .
Auth	Optional key.

POST /v1/simulate/portfolio

Purpose	Joint Monte Carlo for ≥ 2 assets (same shock draw per scenario across names).
Body	PortfolioSimulationRequest : shockpack or catalogue id; portfolio_id , portfolio_assumption_set_id , assets (array length ≥ 2); optional performance_profile .
Response	PortfolioSimulationResult : metadata (includes factor_order , factor_correlation , copula from the resolved shock pack), per_asset[] (same shape as single-asset metrics), portfolio — jointly computed pool metrics (§IV.6.3.1).
Usage	Portfolio concentration; joint breach $\neq$ sum of silo runs; compare weakest-link vs blended vs max DSCR paths and blend IRR tails on portfolio .
Auth	Optional key.

WebSocket WS /ws/v1/simulate/portfolio

Purpose	Same as POST /v1/simulate/portfolio with optional progress messages.
Handshake	If AZRAQ_API_KEY set: header x-api-key .
Messages	Client sends one JSON = PortfolioSimulationRequest . Server: optional { "type": "progress", ... }, then { "type": "result", "body": <PortfolioSimulationResult> } or { "type": "error", "detail" }.
Usage	Long runs from browser / clients that prefer streaming.
Auth	x-api-key when server key is set.

POST /v1/shockpack/catalog/register

Purpose	Store a ShockPackSpec in the catalogue for reuse.
----------------	--

Body	<code>ShockPackSpec</code> (JSON).
Query	<code>semver</code> (default <code>1.0.0</code>), optional <code>tenant_id</code> , <code>promotion_tier</code> (<code>dev</code> / <code>staging</code> / <code>prod</code>), <code>rbac_owner_role</code> .
Response	<code>{ "entry_id": "<uuid>", "response_help": ... }</code> — use <code>entry_id</code> as <code>shockpack_catalog_entry_id</code> on simulate.
Usage	Freeze governance-approved shock design; inline <code>shockpack</code> can still patch fields.
Auth	Optional key.

POST `/v1/shockpack/catalog/{entry_id}/promote`

Purpose	Change catalogue <code>promotion_tier</code> (e.g. <code>dev</code> → <code>prod</code>).
Path	<code>entry_id</code> .
Query	<code>to_tier</code> — <code>dev</code> <code>staging</code> <code>prod</code> .
Headers	<code>X-Azraq-Catalog-Role</code> unless promoter env list is empty.
Response	<code>{ "ok": true, "response_help": ... }</code> — <code>403</code> if role not allowed.
Usage	Gate “approved” packs in <code>GET /v1/shockpack/catalog?promotion_tier=prod</code> .
Auth	Optional key + promoter role.

GET `/v1/shockpack/catalog/{entry_id}`

Purpose	Fetch one catalogue row including full <code>spec</code> .
Path	<code>entry_id</code> .
Headers	Optional <code>X-Azraq-Tenant-Id</code> — must match entry tenant if enforced.
Response	Catalogue object + <code>response_help</code> . <code>403</code> tenant mismatch; <code>404</code> missing.
Usage	Inspect exact shock JSON before production runs.
Auth	Optional key.

GET `/v1/shockpack/catalog`

Purpose	List recent registrations (<code>metadata</code> ; use <code>GET by id</code> for full <code>spec</code>).
Query	<code>limit</code> (default 100), optional <code>tenant_id</code> , <code>promotion_tier</code> . Header tenant may filter.
Response	<code>{ "entries": [...], "response_help": ... }</code> .
Usage	Discover <code>entry_id</code> for CI/CD.
Auth	Optional key.

POST `/v1/shockpack/export/npz`

Purpose	Write correlated shock array <code>Z</code> to a <code>.npz</code> file on the server (offline HPC / custom models).
----------------	--

Body	ShockExportRequest : shockpack (required), optional directory .
Response	{ "path": "<absolute .npz path>", "response_help": ... }. dynamic_margins resolved before draw.
Usage	No DSCR/IRR — only random draws. Default dir: AZRAQ_SHOCK_EXPORT_DIR or data/shockpacks .
Auth	Optional key.

POST /v1/snapshots/save

Purpose	Persist an already computed result to JSON on disk.
Body	SnapshotSaveRequest : optional label , result = full SimulationResult / PortfolioSimulationResult / BaseCaseResult .
Response	{ "path": "<absolute path>", "response_help": ... } .
Usage	Bookmark or feed external tools; same root as scheduled snapshots.
Auth	Optional key.

GET /v1/snapshots/list

Purpose	List snapshot file paths under the snapshot root.
Parameters	None.
Response	{ "paths": ["... "], "response_help": ... } .
Usage	Pick before_path / after_path for diff.
Auth	Optional key.

POST /v1/snapshots/diff/asset-metrics

Purpose	Compare two saved asset SimulationResult snapshots.
Body	{ "before_path": "<server path>", "after_path": "<server path>" } .
Response	metrics_delta , provenance_delta , response_help .
Usage	Explain drift: assumptions vs shock vs seed vs model version. 400 if either file is not an asset MC result.
Auth	Optional key.

GET /v1/audit/runs

Purpose	Recent simulation events from SQLite audit DB (if enabled).
Query	limit (default 50).
Response	{ "runs": [{ "run_id", "run_kind", "asset_id", "shockpack_id", "seed", "n_scenarios", ... }], "response_help": ... } .
Usage	Correlate run_id with SimulationResult.metadata.run_id . Empty if audit disabled/missing.
Auth	Optional key.

IV.5 Errors

HTTP	Typical cause
401	Missing or wrong <code>X-API-Key</code> when server requires it.
403	Catalogue tenant / promoter role mismatch.
404	Yahoo history missing; unknown catalogue <code>entry_id</code> .
400	Validation: bad JSON shape, wrong asset count for portfolio, wrong snapshot types for diff.
422	Pydantic validation detail (FastAPI) — check <code>detail</code> array for field paths.
501	Optional dependency missing (e.g. <code>yfinance</code>).
502	Upstream feed failure (e.g. ECB).

For 422, use `/docs` “Try it out” or client validation against `/openapi.json` to align payloads before production.

IV.6 Live deployment, examples, and response detail

IV.6.1 Public host (Octalooop)

Resource	URL
Swagger UI (schemas + “Try it out”)	<code>https://monte-carlo.octalooop.dev/docs#/</code>
OpenAPI JSON (import to Postman, codegen)	<code>https://monte-carlo.octalooop.dev/openapi.json</code>
Web app (if deployed)	<code>https://monte-carlo.octalooop.dev/app/</code>

Use `https://monte-carlo.octalooop.dev` as the host for the paths in §IV.4 (e.g. `POST https://monte-carlo.octalooop.dev/v1/simulate/asset`). If the server has `AZRAQ_API_KEY` set, add header `X-API-Key` on calls other than `GET /health`.

IV.6.2 `AdhocSimulationRequest` — request fields (main Monte Carlo)

Field	Required	Type / notes
<code>asset</code>	Yes	Full <code>AssetAssumptions</code> (ids, horizon, revenue, opex, capex, equity, financing, optional <code>utility_opex_annual</code> , <code>factor_transforms</code> , <code>full_stack</code> , ...).
<code>shockpack</code>	One of pack or catalogue	Inline <code>ShockPackSpec</code> (<code>shockpack_id</code> , <code>seed</code> , <code>n_scenarios</code> , <code>sampling_method</code> , <code>margins</code> , optional <code>dynamic_margins</code> , <code>correlation</code> , ...).
<code>shockpack_catalog_entry_id</code>	One of pack or catalogue	UUID from <code>POST /v1/shockpack/catalog/register</code> ; inline <code>shockpack</code> can patch fields.
<code>include_attribution</code>	No	Default <code>false</code> . Tail attribution (indicative narrative).
<code>include_advanced_attribution</code>	No	Default <code>false</code> . Extra factor weights when attribution on.
<code>attribution_tail_fraction</code>	No	Default <code>0.05</code> (e.g. worst ~5% scenarios for attribution).
<code>performance_profile</code>	No	"interactive" (caps <code>n_scenarios</code> , good for UI), "standard", "deep" (no cap), or <code>null</code> .

IV.6.3 SimulationResult.metrics — main outputs (read after POST /v1/simulate/asset)

Field	Meaning
dscr	DistributionSummary : percentiles (e.g. p05 – p95), mean , std — debt service coverage over the run.
irr_annual	DistributionSummary for equity IRR (decimal per year, e.g. 0.08 = 8%).
total_capex	Optional DistributionSummary for stochastic nominal build cost when the engine supplies it.
covenant_breach_probability	Share of paths where modeled DSCR falls below asset.financing.covenant_dscr .
probability_of_default_proxy_dscr_lt_1	PD-style proxy using DSCR < 1 (see OpenAPI / product definition).
var_irr_95 / cvar_irr_95	Tail / shortfall style metrics on IRR when populated.
ebitda, levered_cf, nav_proxy_equity, ...	Other DistributionSummary blocks when enabled by model path.

Always read **metadata** for **run_id**, **seed**, **n_scenarios**, **compute_time_ms**, **shockpack_id**, optional **margin_calibration_trace** (if Yahoo/file/http calibration ran).

IV.6.3.1 PortfolioSimulationResult — portfolio object (PortfolioMetrics)

per_asset[i].metrics is the same family as **POST /v1/simulate/asset** . **portfolio** adds **whole-book views** derived from **the same Monte Carlo paths** (one shock draw index shared across assets).

Field	Meaning
probability_any_covenant_breach	Share of scenarios where at least one asset breaches its covenant.
probability_at_least_k_breaches	"2" , "3" , ... for multiple simultaneous breaches ($k \geq 2$).
min_dscr_across_assets	DistributionSummary : each scenario uses the minimum path DSCR across assets (weakest-link view).
max_dscr_across_assets	DistributionSummary : each scenario uses the maximum path DSCR across assets (strongest name).
revenue_weighted_mean_dscr_across_assets	DistributionSummary : each scenario $\sum w_i \cdot \text{DSCR}_i$ with $w_i \propto \text{base_revenue_annual}$ — blend at the portfolio level (not a merged waterfall statement). Same weights underlie weighted_covenant_breach_exposure / revenue_herfindahl .
revenue_weighted_mean_equity_irr_across_assets	DistributionSummary : revenue-weighted mean equity IRR paths (blend; not consolidated-fund IRR from pooled cashflows).
var_irr_95 / cvar_irr_95	Same conventions as SimulationResult.metrics : IRR tail shortfall (median – p05) and mean IRR in worst ~5% on the revenue-weighted blend IRR series.
sum_levered_cf_year1	DistributionSummary of summed year-1 after-tax levered CF across assets.
var_sum_levered_cf_p05 / cvar_sum_levered_cf_p05	Raw p05 summed CF level and mean CF in scenarios at/below it (portfolio cashflow downside).

Field	Meaning
revenue_herfindahl	$\sum w_i^2$ concentration on revenue weights w_i .
weighted_covenant_breach_exposure	$E[\sum w_i \cdot 1(\text{breach}_i)]$ — revenue-weighted covenant stress simultaneously across names.
cross_asset_dscr_correlation_pearson	Symmetric Pearson ρ of scenario-by-scenario DSCR paths (row/column order matches <code>metadata.asset_ids</code>). Output co-movement, not the 4×4 factor <code>correlation</code> input matrix.
cross_asset_equity_irr_correlation_pearson	Same for IRR paths (order matches <code>metadata.asset_ids</code>).

Macro **factor ρ** assumed for shocks appears on `PortfolioRunMetadata` as `factor_order`, `factor_correlation`, `copula`.

IV.6.4 Example: health check (no API key)

Replace `HOST` with `https://monte-carlo.octalloop.dev` OR `http://127.0.0.1:8000`.

```
curl -sS "${HOST}/health"
```

IV.6.5 Example: single-asset Monte Carlo (minimal body)

Send `Content-Type: application/json`. If an API key is required, add `-H "X-API-Key: YOUR_KEY"`.

```
curl -sS -X POST "${HOST}/v1/simulate/asset" \
-H "Content-Type: application/json" \
-d '{
  "shockpack": {
    "shockpack_id": "demo-sp-1",
    "seed": 42,
    "n_scenarios": 800,
    "sampling_method": "monte_carlo",
    "margins": {
      "revenue_log_mean": 0,
      "revenue_log_sigma": 0.08,
      "capex_log_mean": 0,
      "capex_log_sigma": 0.06,
      "opex_log_mean": 0,
      "opex_log_sigma": 0.05,
      "rate_shock_sigma": 0.005
    }
  },
  "asset": {
    "asset_id": "demo-asset-1",
    "assumption_set_id": "demo-as-1",
    "horizon_years": 8,
    "base_revenue_annual": 12000000,
    "base_opex_annual": 5000000,
    "initial_capex": 8000000,
    "equity_fraction": 0.35,
    "tax_rate": 0.0,
    "financing": {
      "debt_principal": 4000000,
      "interest_rate_annual": 0.055,
      "loan_term_years": 12,
      "covenant_dscr": 1.2
    }
  },
  "include_attribution": false,
  "performance_profile": "interactive"
}'
```


Note: The JSON above uses a fully specified `margins` block. If `dynamic_margins` (Yahoo/file/http) is used instead, run `POST /v1/calibration/preview` first to validate resolved sigmas. Omit `performance_profile` or use `"standard"` / `"deep"` for higher scenario limits (see OpenAPI).

IV.6.6 Example: deterministic baseline (no Monte Carlo)

Body is `AssetAssumptions` only (not wrapped in `asset`).

```
curl -sS -X POST "${HOST}/v1/simulate/v0/base-case" \
-H "Content-Type: application/json" \
-d '{"asset_id":"demo","assumption_set_id":"v0","horizon_years":8,
    "base_revenue_annual":12000000,"base_opex_annual":5000000,
    "initial_capex":80000000,"equity_fraction":0.35,"tax_rate":0,
    "financing":{"debt_principal":40000000,"interest_rate_annual":0.055,
    "loan_term_years":12,"covenant_dscr":1.2}}'
```

IV.6.7 WebSocket portfolio run

Connect to `wss://monte-carlo.octaloop.dev/ws/v1/simulate/portfolio` (or `ws://127.0.0.1:8000/...` locally). Send **one** message: same JSON as `POST /v1/simulate/portfolio`. If API key is set, send header `x-api-key` on the handshake.

IV.6.8 Where to see every field

Open [Swagger UI](#), expand `POST /v1/simulate/asset`, inspect **Request body** and **Response** schemas (`AdhocSimulationRequest`, `SimulationResult`, `FinancialRiskMetrics`, ...). The live schema explorer is authoritative for nested optional fields.

End of complete guide.